

techniques have become popular only in the last decade, they are basically variants of “top-down parsing languages” described in the early 1970.

3.11 Exercises

The main goal of these exercises is to familiarize yourself with regular expressions, automata, grammars, the `fslex` lexer generator and the `fsyacc` parser generator.

Exercise 3.1 Do exercises 1.2 and 1.3 in Mogensen’s book [12].

Exercise 3.2 Write a regular expression that recognizes all sequences consisting of a and b where two a ’s are always separated by at least one b . For instance, these four strings are legal: b , a , ba , $ababbbaba$; but these two strings are illegal: aa , $babaa$.

Construct the corresponding NFA. Try to find a DFA corresponding to the NFA.

Exercise 3.3 Write out the rightmost derivation of the string below from the expression grammar at the end of Sect. 3.6.5, corresponding to `ExprPar.fsy`. Take note of the sequence of grammar rules (A–I) used.

```
let z = (17) in z + 2 * 3 end EOF
```

Exercise 3.4 Draw the above derivation as a tree.

Exercise 3.5 Get `expr.zip` from the book homepage and unpack it. Using a command prompt, generate (1) the lexer and (2) the parser for expressions by running `fslex` and `fsyacc`; then (3) load the expression abstract syntax, the lexer and parser modules, and the expression interpreter and compilers, into an interactive F# session (`fsharp`):

```
fslex --unicode ExprLex.fsl
fsyacc --module ExprPar ExprPar.fsy
fsharp -r FSharp.PowerPack.dll Absyn.fs ExprPar.fs ExprLex.fs ^
    Parse.fs
```

Now try the parser on several example expressions, both well-formed and ill-formed ones, such as these, and some of your own invention:

```
open Parse;;
fromString "1 + 2 * 3";;
fromString "1 - 2 - 3";;
fromString "1 + -2";;
fromString "x++";;
fromString "1 + 1.2";;
fromString "1 + ";;
fromString "let z = (17) in z + 2 * 3 end";;
fromString "let z = 17) in z + 2 * 3 end";;
fromString "let in = (17) in z + 2 * 3 end";;
fromString "1 + let x=5 in let y=7+x in y+y end + x end";;
```

Exercise 3.6 Use the expression parser from `Parse.fs` and the compiler `scomp` (from expressions to stack machine instructions) and the associated datatypes from `Expr.fs`, to define a function `compString : string -> sinstr list` that parses a string as an expression and compiles it to stack machine code.

Exercise 3.7 Extend the expression language abstract syntax and the lexer and parser specifications with conditional expressions. The abstract syntax should be `If(e1, e2, e3)`, so modify file `Absyn.fs` as well as `ExprLex.fsl` and file `ExprPar.fsy`. The concrete syntax may be the keyword-laden F#/ML-style:

```
if e1 then e2 else e3
```

or the more light-weight C/C++/Java/C#-style:

```
e1 ? e2 : e3
```

Some documentation for `fslex` and `fsyacc` is found in this chapter and in *Expert F#* [17].

Exercise 3.8 Consider the parser generated from `ExprPar.fsy`, and determine what steps are taken during the parsing of this string:

```
let z = (17) in z + 2 * 3 end EOF
```

For each step, show the remaining input, the parse stack, and the action (shift, reduce, or goto) performed. You will need a printout of the parser states and their transitions in `ExprPar.fsyacc.output` to do this exercise. Sanity check: the sequence of reduce action rule numbers in the parse should be the exact reverse of that found in the derivation in Exercise 3.3.

Exercise 3.9 Files in the subdirectory `Usql/` contain abstract syntax abstract syntax (file `Absyn.fs`), an informal grammar (file `grammar.txt`), a lexer specification (`UsqlLex.fsl`) and a parser specification (`UsqlPar.fsy`) for micro-SQL, a small subset of the SQL database query language.

Extend micro-SQL to cover a larger class of SQL `SELECT` statements. Look at the examples below and decide your level of ambition. You should not need to modify file `Parse.fs`. Don't forget to write some examples in concrete syntax to show that your parser can parse them.

For instance, to permit an optional `WHERE` clause, you may add one more components to the `Select` constructor:

```
type stmt =
| Select of expr list           (* fields are expressions *)
          * string list        (* FROM ... *)
          * expr option         (* optional WHERE clause *)
```

so that `SELECT...FROM...WHERE...` gives `Select(..., ..., SOME ...)`, and `SELECT...FROM...` gives `Select(..., ..., NONE)`.